```
1  template <typename Container>
2  void removeEveryOtherItem( Container & lst )
3  {
4      auto itr = lst.begin( );     // itr is a Container::iterator
5
6      while( itr != lst.end( ) )
7      {
8          itr = lst.erase( itr );
9          if( itr != lst.end( ) )
10             ++itr;
11     }
12 }
```

Figure 3.5 Using iterators to remove every other item in a List (either a `vector` or `list`). Efficient for a `list`, but not for a `vector`.

timing information. The function template is shown in Figure 3.5. The use of `auto` at line 4 is a C++11 feature that allows us to avoid the longer type `Container::iterator`. If we run the code, passing a `list<int>`, it takes 0.039 sec for a 800,000-item `list`, and 0.073 sec for an 1,600,000-item `list`, and is clearly a linear-time routine, because the running time increases by the same factor as the input size. When we pass a `vector<int>`, the routine takes almost five minutes for an 800,000-item `vector` and about twenty minutes for an 1,600,000-item `vector`; the four fold increase in running time when the input increases by only a factor of two is consistent with quadratic behavior.

3.3.3 const_iterators

The result of `*itr` is not just the value of the item that the iterator is viewing but also the item itself. This distinction makes the iterators very powerful but also introduces some complications. To see the benefit, suppose we want to change all the items in a collection to a specified value. The following routine works for both `vector` and `list` and runs in linear time. It's a wonderful example of writing generic, type-independent code.

```
template <typename Container, typename Object>
void change( Container & c, const Object & newValue )
{
    typename Container::iterator itr = c.begin( );
    while( itr != c.end( ) )
        *itr++ = newValue;
}
```

To see the potential problem, suppose the `Container c` was passed to a routine using call-by-constant reference. This means we would expect that no changes would be allowed to `c`, and the compiler would ensure this by not allowing calls to any of `c`'s mutators. Consider the following code that prints a `list` of integers but also tries to sneak in a change to the `list`:

```
void print( const list<int> & lst, ostream & out = cout )
{
    typename Container::iterator itr = lst.begin( );
    while( itr != lst.end( ) )
    {
        out << *itr << endl;
        *itr = 0;   // This is fishy!!!
        ++itr;
    }
}
```

If this code were legal, then the const-ness of the `list` would be completely meaningless, because it would be so easily bypassed. The code is not legal and will not compile. The solution provided by the STL is that every collection contains not only an `iterator` nested type but also a `const_iterator` nested type. The main difference between an `iterator` and a `const_iterator` is that `operator*` for `const_iterator` returns a constant reference, and thus `*itr` for a `const_iterator` cannot appear on the left-hand side of an assignment statement.

Further, the compiler will force you to use a `const_iterator` to traverse a constant collection. It does so by providing two versions of `begin` and two versions of `end`, as follows:

- `iterator begin( )`
- `const_iterator begin( ) const`
- `iterator end( )`
- `const_iterator end( ) const`

The two versions of `begin` can be in the same class only because the const-ness of a method (i.e., whether it is an accessor or mutator) is considered to be part of the signature. We saw this trick in Section 1.7.2 and we will see it again in Section 3.4, both in the context of overloading `operator[]`.

If `begin` is invoked on a nonconstant container, the "mutator" version that returns an `iterator` is invoked. However, if `begin` is invoked on a constant container, what is returned is a `const_iterator`, and the return value may not be assigned to an `iterator`. If you try to do so, a compiler error is generated. Once `itr` is a `const_iterator`, `*itr=0` is easily detected as being illegal.

If you use `auto` to declare your iterators, the compiler will deduce for you whether an `iterator` or `const_iterator` is substituted; to a large extent, this relieves the programmer from having to keep track of the correct iterator type and is precisely one of the intended uses of `auto`. Additionally, library classes such as `vector` and `list` that provide iterators as described above are compatible with the range-based `for` loop, as are user-defined classes.

An additional feature in C++11 allows one to write code that works even if the `Container` type does not have `begin` and `end` member functions. Non-member free functions `begin` and `end` are defined that allow one to use `begin(c)` in any place where `c.begin()` is allowed. Writing generic code using `begin(c)` instead of `c.begin()` has the advantage that it allows the generic code to work on containers that have `begin/end` as members, as well as those that do not have `begin/end` but which can later be augmented with appropriate

```
template <typename Container>
void print( const Container & c, ostream & out = cout )
{
    if( c.empty( ) )
        out << "(empty)";
    else
    {
        auto itr = begin( c );   // itr is a Container::const_iterator

        out << "[ " << *itr++;   // Print first item

        while( itr != end( c ) )
            out << ", " << *itr++;
        out << " ]" << endl;
    }
}
```

Figure 3.6 Printing any container

non-member functions. The addition of `begin` and `end` as free functions in C++11 is made possible by the addition of language features `auto` and `decltype`, as shown in the code below.

```
template<typename Container>
auto begin( Container & c ) -> decltype( c.begin( ) )
{
    return c.begin( );
}

template<typename Container>
auto begin( const Container & c ) -> decltype( c.begin( ) )
{
    return c.begin( );
}
```

In this code, the return type of `begin` is deduced to be the type of `c.begin()` .

The code in Figure 3.6 makes use of `auto` to declare the iterator (as in Fig. 3.5) and uses non-member functions `begin` and `end`.

3.4 Implementation of `vector`

In this section, we provide the implementation of a usable `vector` class template. The `vector` will be a first-class type, meaning that unlike the primitive array in C++, the `vector` can be copied, and the memory it uses can be automatically reclaimed (via its destructor). In Section 1.5.7, we described some important features of C++ primitive arrays:

- The array is simply a pointer variable to a block of memory; the actual array size must be maintained separately by the programmer.
- The block of memory can be allocated via `new[]` but then must be freed via `delete[]`.
- The block of memory cannot be resized (but a new, presumably larger block can be obtained and initialized with the old block, and then the old block can be freed).

To avoid ambiguities with the library class, we will name our class template `Vector`. Before examining the `Vector` code, we outline the main details:

1. The `Vector` will maintain the primitive array (via a pointer variable to the block of allocated memory), the array capacity, and the current number of items stored in the `Vector`.
2. The `Vector` will implement the Big-Five to provide deep-copy semantics for the copy constructor and `operator=`, and will provide a destructor to reclaim the primitive array. It will also implement C++11 move semantics.
3. The `Vector` will provide a `resize` routine that will change (generally to a larger number) the size of the `Vector` and a `reserve` routine that will change (generally to a larger number) the capacity of the `Vector`. The capacity is changed by obtaining a new block of memory for the primitive array, copying the old block into the new block, and reclaiming the old block.
4. The `Vector` will provide an implementation of `operator[]` (as mentioned in Section 1.7.2, `operator[]` is typically implemented with both an accessor and mutator version).
5. The `Vector` will provide basic routines, such as `size`, `empty`, `clear` (which are typically one-liners), `back`, `pop_back`, and `push_back`. The `push_back` routine will call `reserve` if the size and capacity are same.
6. The `Vector` will provide support for the nested types `iterator` and `const_iterator`, and associated `begin` and `end` methods.

Figure 3.7 and Figure 3.8 show the `Vector` class. Like its STL counterpart, there is limited error checking. Later we will briefly discuss how error checking can be provided.

As shown on lines 118 to 120, the `Vector` stores the size, capacity, and primitive array as its data members. The constructor at lines 7 to 9 allows the user to specify an initial size, which defaults to zero. It then initializes the data members, with the capacity slightly larger than the size, so a few `push_back`s can be performed without changing the capacity.

The copy constructor, shown at lines 11 to 17, makes a new `Vector` and can then be used by a casual implementation of `operator=` that uses the standard idiom of swapping in a copy. This idiom works only if swapping is done by moving, which itself requires the implementation of the move constructor and move `operator=` shown at lines 29 to 44. Again, these use very standard idioms. Implementation of the copy assignment `operator=` using a copy constructor and swap, while simple, is certainly not the most efficient method, especially in the case where both `Vector`s have the same size. In that special case, which can be tested for, it can be more efficient to simply copy each element one by one using `Object`'s `operator=`.

```
#include <algorithm>

template <typename Object>
class Vector
{
  public:
    explicit Vector( int initSize = 0 ) : theSize{ initSize },
         theCapacity{ initSize + SPARE_CAPACITY }
      { objects = new Object[ theCapacity ]; }

    Vector( const Vector & rhs ) : theSize{ rhs.theSize },
        theCapacity{ rhs.theCapacity }, objects{ nullptr }
    {
        objects = new Object[ theCapacity  ];
        for( int k = 0; k < theSize; ++k )
            objects[ k ] = rhs.objects[ k ];
    }

    Vector & operator= ( const Vector & rhs )
    {
        Vector copy = rhs;
        std::swap( *this, copy );
        return *this;
    }

    ~Vector( )
      { delete [ ] objects; }

    Vector( Vector && rhs ) : theSize{ rhs.theSize },
        theCapacity{ rhs.theCapacity }, objects{ rhs.objects }
    {
        rhs.objects = nullptr;
        rhs.theSize = 0;
        rhs.theCapacity = 0;
    }

    Vector & operator= ( Vector && rhs )
    {
        std::swap( theSize, rhs.theSize );
        std::swap( theCapacity, rhs.theCapacity );
        std::swap( objects, rhs.objects );

        return *this;
    }

```

Figure 3.7 `vector` class (Part 1 of 2)

```
46      void resize( int newSize )
47      {
48          if( newSize > theCapacity )
49              reserve( newSize * 2 );
50          theSize = newSize;
51      }
52
53      void reserve( int newCapacity )
54      {
55          if( newCapacity < theSize )
56              return;
57
58          Object *newArray = new Object[ newCapacity ];
59          for( int k = 0; k < theSize; ++k )
60              newArray[ k ] = std::move( objects[ k ] );
61
62          theCapacity = newCapacity;
63          std::swap( objects, newArray );
64          delete [ ] newArray;
65      }
```

Figure 3.7 *(continued)*

The `resize` routine is shown at lines 46 to 51. The code simply sets the `theSize` data member, after possibly expanding the capacity. Expanding capacity is very expensive. So if the capacity is expanded, it is made twice as large as the size to avoid having to change the capacity again unless the size increases dramatically (the `+1` is used in case the size is 0). Expanding capacity is done by the `reserve` routine, shown at lines 53 to 65. It consists of allocation of a new array at line 58, moving the old contents at lines 59 and 60, and the reclaiming of the old array at line 64. As shown at lines 55 and 56, the `reserve` routine can also be used to shrink the underlying array, but only if the specified new capacity is at least as large as the size. If it isn't, the `reserve` request is ignored.

The two versions of `operator[]` are trivial (and in fact very similar to the implementations of `operator[]` in the `matrix` class in Section 1.7.2) and are shown in lines 67 to 70. Error checking is easily added by making sure that `index` is in the range `0` to `size()-1`, inclusive, and throwing an exception if it is not.

A host of short routines, namely, `empty`, `size`, `capacity`, `push_back`, `pop_back`, and `back`, are implemented in lines 72 to 101. At lines 83 and 90, we see the use of the postfix `++` operator, which uses `theSize` to index the array and then increases `theSize`. We saw the same idiom when discussing iterators: `*itr++` uses `itr` to decide which item to view and then advances `itr`. The positioning of the `++` matters: In the prefix `++` operator, `*++itr` advances `itr` and then uses the new `itr` to decide which item to view, and likewise, `objects[++theSize]` would increment `theSize` and use the new value to index the array (which is not what we would want). `pop_back` and `back` could both benefit from error checks in which an exception is thrown if the size is 0.

```
    Object & operator[]( int index )
      { return objects[ index ]; }
    const Object & operator[]( int index ) const
      { return objects[ index ]; }

    bool empty( ) const
      { return size( ) == 0; }
    int size( ) const
      { return theSize; }
    int capacity( ) const
      { return theCapacity; }

    void push_back( const Object & x )
    {
        if( theSize == theCapacity )
            reserve( 2 * theCapacity + 1 );
        objects[ theSize++ ] = x;
    }

    void push_back( Object && x )
    {
        if( theSize == theCapacity )
            reserve( 2 * theCapacity + 1 );
        objects[ theSize++ ] = std::move( x );
    }

    void pop_back( )
    {
        --theSize;
    }

    const Object & back ( ) const
    {
         return objects[ theSize - 1 ];
    }

    typedef Object * iterator;
    typedef const Object * const_iterator;

    iterator begin( )
      { return &objects[ 0 ]; }
    const_iterator begin( ) const
      { return &objects[ 0 ]; }
```

Figure 3.8 `vector` class (Part 2 of 2)

```
110       iterator end( )
111         { return &objects[ size( ) ]; }
112       const_iterator end( ) const
113         { return &objects[ size( ) ]; }
114
115       static const int SPARE_CAPACITY = 16;
116
117     private:
118       int theSize;
119       int theCapacity;
120       Object * objects;
121   };
```

Figure 3.8 *(continued)*

Finally, at lines 103 to 113 we see the declaration of the `iterator` and `const_iterator` nested types and the two `begin` and two `end` methods. This code makes use of the fact that in C++, a pointer variable has all the same operators that we expect for an `iterator`. Pointer variables can be copied and compared; the `*` operator yields the object being pointed at, and, most peculiarly, when `++` is applied to a pointer variable, the pointer variable then points at the object that would be stored next sequentially: If the pointer is pointing inside an array, incrementing the pointer positions it at the next array element. These semantics for pointers date back to the early 70s with the C programming language, upon which C++ is based. The STL iterator mechanism was designed in part to mimic pointer operations.

Consequently, at lines 103 and 104, we see `typedef` statements that state the `iterator` and `const_iterator` are simply other names for a pointer variable, and `begin` and `end` need to simply return the memory addresses representing the first array position and the first invalid array position, respectively.

The correspondence between iterators and pointers for the `vector` type means that using a `vector` instead of the C++ array is likely to carry little overhead. The disadvantage is that, as written, the code has no error checks. If the iterator `itr` goes crashing past the end marker, neither `++itr` nor `*itr` will necessarily signal an error. To fix this problem would require that the `iterator` and `const_iterator` be actual nested class types rather than simply pointer variables. Using nested class types is much more common and is what we will see in the `List` class in Section 3.5.

3.5 Implementation of `list`

In this section, we provide the implementation of a usable `list` class template. As in the case of the `vector` class, our list class will be named `List` to avoid ambiguities with the library class.

Recall that the `List` class will be implemented as a doubly linked list and that we will need to maintain pointers to both ends of the list. Doing so allows us to maintain constant time cost per operation, so long as the operation occurs at a known position. The known position can be at either end or at a position specified by an iterator.

In considering the design, we will need to provide four classes:

1. The `List` class itself, which contains links to both ends, the size of the list, and a host of methods.
2. The `Node` class, which is likely to be a private nested class. A node contains the data and pointers to the previous and next nodes, along with appropriate constructors.
3. The `const_iterator` class, which abstracts the notion of a position, and is a public nested class. The `const_iterator` stores a pointer to "current" node, and provides implementation of the basic iterator operations, all in the form of overloaded operators such as `=`, `==`, `!=`, and `++`.
4. The `iterator` class, which abstracts the notion of a position, and is a public nested class. The `iterator` has the same functionality as `const_iterator`, except that `operator*` returns a reference to the item being viewed, rather than a constant reference to the item. An important technical issue is that an `iterator` can be used in any routine that requires a `const_iterator`, but not vice versa. In other words, `iterator` *IS-A* `const_iterator`.

Because the iterator classes store a pointer to the "current node," and the end marker is a valid position, it makes sense to create an extra node at the end of the list to represent the endmarker. Further, we can create an extra node at the front of the list, logically representing the beginning marker. These extra nodes are sometimes known as **sentinel nodes**; specifically, the node at the front is sometimes known as a **header node**, and the node at the end is sometimes known as a **tail node**.

The advantage of using these extra nodes is that they greatly simplify the coding by removing a host of special cases. For instance, if we do not use a header node, then removing the first node becomes a special case, because we must reset the list's link to the first node during the remove and because the remove algorithm in general needs to access the node prior to the node being removed (and without a header node, the first node does not have a node prior to it). Figure 3.9 shows a doubly linked list with header and tail nodes. Figure 3.10 shows an empty list.

Figure 3.11 and Figure 3.12 show the outline and partial implementation of the `List` class.

We can see at line 5 the beginning of the declaration of the private nested `Node` class. Rather than using the `class` keyword, we use `struct`. In C++, the `struct` is a relic from the C programming language. A `struct` in C++ is essentially a `class` in which the members default to public. Recall that in a `class`, the members default to private. Clearly the `struct`

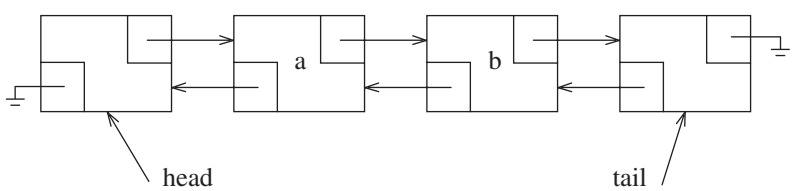

Figure 3.9 A doubly linked list with header and tail nodes

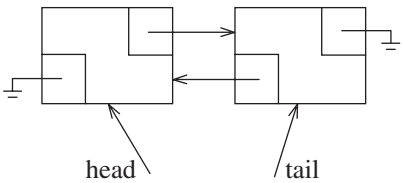

Figure 3.10 An empty doubly linked list with header and tail nodes

keyword is not needed, but you will often see it and it is commonly used by programmers to signify a type that contains mostly data that are accessed directly, rather than through methods. In our case, making the members public in the `Node` class will not be a problem, since the `Node` class is itself private and inaccessible outside of the `List` class.

At line 9 we see the beginning of the declaration of the public nested `const_iterator` class, and at line 12 we see the beginning of the declaration of the public nested `iterator` class. The unusual syntax is **inheritance**, which is a powerful construct not otherwise used in the book. The inheritance syntax states that `iterator` has exactly the same functionality as `const_iterator`, with possibly some additions, and that `iterator` is type-compatible with `const_iterator` and can be used wherever `const_iterator` is needed. We'll discuss those details when we see the actual implementations later.

Lines 80 to 82 contain the data members for `List`, namely, the pointers to the header and tail nodes. We also keep track of the size in a data member so that the `size` method can be implemented in constant time.

The rest of the `List` class consists of the constructor, the Big-Five, and a host of methods. Many of the methods are one-liners. `begin` and `end` return appropriate iterators; the call at line 30 is typical of the implementation, in which we return a constructed iterator (thus the `iterator` and `const_iterator` classes each have a constructor that takes a pointer to a `Node` as its parameter).

The `clear` method at lines 43 to 47 works by repeatedly removing items until the `List` is empty. Using this strategy allows `clear` to avoid getting its hands dirty reclaiming nodes because the node reclamation is now funneled to `pop_front`. The methods at lines 48 to 67 all work by cleverly obtaining and using an appropriate iterator. Recall that the `insert` method inserts prior to a position, so `push_back` inserts prior to the endmarker, as required. In `pop_back`, note that `erase(-end())` creates a temporary iterator corresponding to the endmarker, retreats the temporary iterator, and uses that iterator to `erase`. Similar behavior occurs in `back`. Note also that in the case of the `pop_front` and `pop_back` operations, we again avoid dealing with node reclamation.

Figure 3.13 shows the `Node` class, consisting of the stored item, pointers to the previous and next `Node`, and a constructor. All the data members are public.

Figure 3.14 shows the `const_iterator` class, and Figure 3.15 shows the `iterator` class. As we mentioned earlier, the syntax at line 39 in Figure 3.15 indicates an advanced feature known as *inheritance* and means that `iterator` *IS-A* `const_iterator`. When the `iterator` class is written this way, it inherits all the data and methods from `const_iterator`. It may then add new data, add new methods, and override (i.e., redefine) existing methods. In the most general scenario, there is significant syntactical baggage (often resulting in the keyword `virtual` appearing in the code).

```
template <typename Object>
class List
{
  private:
    struct Node
      { /* See Figure 3.13 */ };

  public:
    class const_iterator
      { /* See Figure 3.14 */ };

    class iterator : public const_iterator
      { /* See Figure 3.15 */ };

  public:
    List( )
      { /* See Figure 3.16 */ }
    List( const List & rhs )
      { /* See Figure 3.16 */ }
    ~List( )
      { /* See Figure 3.16 */ }
    List & operator= ( const List & rhs )
      { /* See Figure 3.16 */ }
    List( List && rhs )
      { /* See Figure 3.16 */ }
    List & operator= ( List && rhs )
      { /* See Figure 3.16 */ }

    iterator begin( )
      { return { head->next }; }
    const_iterator begin( ) const
      { return { head->next }; }
    iterator end( )
      { return { tail }; }
    const_iterator end( ) const
      { return { tail }; }

    int size( ) const
      { return theSize; }
    bool empty( ) const
      { return size( ) == 0; }

    void clear( )
    {
        while( !empty( ) )
            pop_front( );
    }
```

Figure 3.11 `List` class (Part 1 of 2)

```
    Object & front( )
      { return *begin( ); }
    const Object & front( ) const
      { return *begin( ); }
    Object & back( )
      { return *--end( ); }
    const Object & back( ) const
      { return *--end( ); }
    void push_front( const Object & x )
      { insert( begin( ), x ); }
    void push_front( Object && x )
      { insert( begin( ), std::move( x ) ); }
    void push_back( const Object & x )
      { insert( end( ), x ); }
    void push_back( Object && x )
      { insert( end( ), std::move( x ) ); }
    void pop_front( )
      { erase( begin( ) ); }
    void pop_back( )
      { erase( --end( ) ); }

    iterator insert( iterator itr, const Object & x )
      { /* See Figure 3.18 */ }
    iterator insert( iterator itr, Object && x )
      { /* See Figure 3.18 */ }

    iterator erase( iterator itr )
      { /* See Figure 3.20 */ }
    iterator erase( iterator from, iterator to )
      { /* See Figure 3.20 */ }

  private:
    int   theSize;
    Node *head;
    Node *tail;

    void init( )
      { /* See Figure 3.16 */ }
};
```

Figure 3.12 List class (Part 2 of 2)

```
struct Node
{
    Object  data;
    Node   *prev;
    Node   *next;

    Node( const Object & d = Object{ }, Node * p = nullptr,
                                        Node * n = nullptr )
      : data{ d }, prev{ p }, next{ n } { }

    Node( Object && d, Node * p = nullptr, Node * n = nullptr )
      : data{ std::move( d ) }, prev{ p }, next{ n } { }
};
```

Figure 3.13 Nested `Node` class for `List` class

However, in our case, we can avoid much of the syntactical baggage because we are not adding new data, nor are we intending to change the behavior of an existing method. We are, however, adding some new methods in the `iterator` class (with very similar signatures to the existing methods in the `const_iterator` class). As a result, we can avoid using `virtual`. Even so, there are quite a few syntax tricks in `const_iterator`.

At lines 28 and 29, `const_iterator` stores as its single data member a pointer to the "current" node. Normally, this would be private, but if it were private, then `iterator` would not have access to it. Marking members of `const_iterator` as `protected` allows the classes that inherit from `const_iterator` to have access to these members, but does not allow other classes to have access.

At lines 34 and 35 we see the constructor for `const_iterator` that was used in the `List` class implementation of `begin` and `end`. We don't want all classes to see this constructor (iterators are not supposed to be visibly constructed from pointer variables), so it can't be public, but we also want the `iterator` class to be able to see it, so logically this constructor is made protected. However, this doesn't give `List` access to the constructor. The solution is the **friend declaration** at line 37, which grants the `List` class access to `const_iterator`'s nonpublic members.

The public methods in `const_iterator` all use operator overloading. `operator==`, `operator!=`, and `operator*` are straightforward. At lines 10 to 21 we see the implementation of `operator++`. Recall that the prefix and postfix versions of `operator++` are completely different in semantics (and precedence), so we need to write separate routines for each form. They have the same name, so they must have different signatures to be distinguished. C++ requires that we give them different signatures by specifying an empty parameter list for the prefix form and a single (anonymous) `int` parameter for the postfix form. Then `++itr` calls the zero-parameter `operator++`; and `itr++` calls the one-parameter `operator++`. The `int` parameter is never used; it is present only to give a different signature. The implementation suggests that, in many cases where there is a choice between using the prefix or postfix `operator++`, the prefix form will be faster than the postfix form.

In the `iterator` class, the protected constructor at line 64 uses an initialization list to initialize the inherited current node. We do not have to reimplement `operator==`

```
class const_iterator
{
  public:
    const_iterator( ) : current{ nullptr }
      { }

    const Object & operator* ( ) const
      { return retrieve( ); }

    const_iterator & operator++ ( )
    {
        current = current->next;
        return *this;
    }

    const_iterator operator++ ( int )
    {
        const_iterator old = *this;
        ++( *this );
        return old;
    }

    bool operator== ( const const_iterator & rhs ) const
      { return current == rhs.current; }
    bool operator!= ( const const_iterator & rhs ) const
      { return !( *this == rhs ); }

  protected:
    Node *current;

    Object & retrieve( ) const
      { return current->data; }

    const_iterator( Node *p ) : current{ p }
      { }

    friend class List<Object>;
};
```

Figure 3.14 Nested `const_iterator` class for `List` class

and `operator!=` because those are inherited unchanged. We do provide a new pair of `operator++` implementations (because of the changed return type) that hide the originals in the `const_iterator`, and we provide an accessor/mutator pair for `operator*`. The accessor `operator*`, shown at lines 47 and 48, simply uses the same implementation as in `const_iterator`. The accessor is explicitly implemented in `iterator` because otherwise the original implementation is hidden by the newly added mutator version.

```
39      class iterator : public const_iterator
40      {
41        public:
42          iterator( )
43            { }
44
45          Object & operator* ( )
46            { return const_iterator::retrieve( ); }
47          const Object & operator* ( ) const
48            { return const_iterator::operator*( ); }
49
50          iterator & operator++ ( )
51          {
52              this->current = this->current->next;
53              return *this;
54          }
55
56          iterator operator++ ( int )
57          {
58              iterator old = *this;
59              ++( *this );
60              return old;
61          }
62
63        protected:
64          iterator( Node *p ) : const_iterator{ p }
65            { }
66
67          friend class List<Object>;
68      };
```

Figure 3.15 Nested `iterator` class for `List` class

Figure 3.16 shows the constructor and Big-Five. Because the zero-parameter constructor and copy constructor must both allocate the header and tail nodes, we provide a private `init` routine. `init` creates an empty `List`. The destructor reclaims the header and tail nodes; all the other nodes are reclaimed when the destructor invokes `clear`. Similarly, the copy constructor is implemented by invoking public methods rather than attempting low-level pointer manipulations.

Figure 3.17 illustrates how a new node containing `x` is spliced in between a node pointed at by `p` and `p.prev`. The assignment to the node pointers can be described as follows:

```
Node *newNode = new Node{ x, p->prev, p };  // Steps 1 and 2
p->prev->next = newNode;                    // Step 3
p->prev = newNode;                          // Step 4
```

```
    List( )
      { init( ); }

    ~List( )
    {
        clear( );
        delete head;
        delete tail;
    }

    List( const List & rhs )
    {
        init( );
        for( auto & x : rhs )
            push_back( x );
    }

    List & operator= ( const List & rhs )
    {
        List copy = rhs;
        std::swap( *this, copy );
        return *this;
    }


    List( List && rhs )
      : theSize{ rhs.theSize }, head{ rhs.head }, tail{ rhs.tail }
    {
        rhs.theSize = 0;
        rhs.head = nullptr;
        rhs.tail = nullptr;
    }

    List & operator= ( List && rhs )
    {
        std::swap( theSize, rhs.theSize );
        std::swap( head, rhs.head );
        std::swap( tail, rhs.tail );

        return *this;
    }

    void init( )
    {
        theSize = 0;
        head = new Node;
        tail = new Node;
        head->next = tail;
        tail->prev = head;
    }
```

Figure 3.16 Constructor, Big-Five, and private `init` routine for `List` class

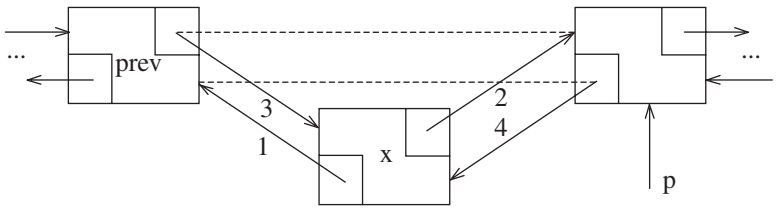

Figure 3.17 Insertion in a doubly linked list by getting a new node and then changing pointers in the order indicated

Steps 3 and 4 can be combined, yielding only two lines:

```
Node *newNode = new Node{ x, p->prev, p };  // Steps 1 and 2
p->prev = p->prev->next = newNode;          // Steps 3 and 4
```

But then these two lines can also be combined, yielding:

```
p->prev = p->prev->next = new Node{ x, p->prev, p };
```

This makes the `insert` routine in Figure 3.18 short.

Figure 3.19 shows the logic of removing a node. If `p` points to the node being removed, only two pointers change before the node can be reclaimed:

```
p->prev->next = p->next;
p->next->prev = p->prev;
delete p;
```

Figure 3.20 shows a pair of `erase` routines. The first version of `erase` contains the three lines of code shown above and the code to return an `iterator` representing the item after

```
// Insert x before itr.
iterator insert( iterator itr, const Object & x )
{
    Node *p = itr.current;
    theSize++;
    return { p->prev = p->prev->next = new Node{ x, p->prev, p } };
}

// Insert x before itr.
iterator insert( iterator itr, Object && x )
{
    Node *p = itr.current;
    theSize++;
    return { p->prev = p->prev->next
                              = new Node{ std::move( x ), p->prev, p } };
}
```

Figure 3.18 `insert` routine for `List` class

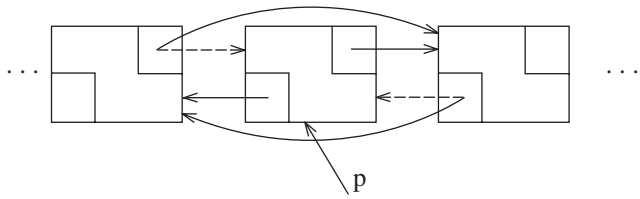

Figure 3.19 Removing node specified by `p` from a doubly linked list

```
1      // Erase item at itr.
2      iterator erase( iterator itr )
3      {
4          Node *p = itr.current;
5          iterator retVal{ p->next };
6          p->prev->next = p->next;
7          p->next->prev = p->prev;
8          delete p;
9          theSize--;
10
11         return retVal;
12     }
13
14     iterator erase( iterator from, iterator to )
15     {
16         for( iterator itr = from; itr != to; )
17             itr = erase( itr );
18
19         return to;
20     }
```

Figure 3.20 `erase` routines for `List` class

the erased element. Like `insert`, `erase` must update `theSize`. The second version of `erase` simply uses an `iterator` to call the first version of `erase`. Note that we cannot simply use `itr++` in the `for` loop at line 16 and ignore the return value of `erase` at line 17. The value of `itr` is stale immediately after the call to `erase`, which is why `erase` returns an `iterator`.

In examining the code, we can see a host of errors that can occur and for which no checks are provided. For instance, iterators passed to `erase` and `insert` can be uninitialized or for the wrong list! Iterators can have `++` or `*` applied to them when they are already at the endmarker or are uninitialized.

An uninitialized iterator will have `current` pointing at `nullptr`, so that condition is easily tested. The endmarker's `next` pointer points at `nullptr`, so testing for `++` or `*` on an endmarker condition is also easy. However, in order to determine if an iterator passed to `erase` or `insert` is an iterator for the correct list, the iterator must store an additional data member representing a pointer to the `List` from which it was constructed.